Experiment 25: Restricted Boltzmann Machine Energy and Reconstruction

Question

Problem Statement: A movie-streaming service wants to learn hidden taste patterns from users' binary watched/not-watched records and fill in likely preferences. A binary user–movie matrix (or binarised MNIST digits) with 784 visible units and 64 hidden units. The objective is to implement an RBM, compute its energy function and reconstruct inputs. Use NumPy, SciPy, Matplotlib.

Tasks:

1. Implement the energy function $E(v, h) = -a^T v - b^T h - v^T W h$ in NumPy.
2. Compute hidden activation probabilities $p(h|v)$ and visible probabilities $p(v|h)$.
3. Train the RBM with one-step contrastive divergence.
4. Plot average free energy and reconstruction error over epochs.
5. Show original and reconstructed samples side by side.

Code

```
"""Experiment 25: Restricted Boltzmann Machine Energy and Reconstruction (binarised MNIST)"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit            # sigmoid
from itertools import product

rng = np.random.RandomState(0)

# ---------------- Data: binarised MNIST (784 visible units) ----------------
def load_mnist(n_train=10000, n_test=1000):
    try:
        from tensorflow import keras
        (x, _), (xt, _) = keras.datasets.mnist.load_data()
        x = x.reshape(-1, 784) / 255.0; xt = xt.reshape(-1, 784) / 255.0
    except Exception as e:                                      # offline fallback (8x8 digits scaled to
28x28)
        print("WARNING: MNIST not available (", type(e).__name__, ") -> using scikit-learn digits scaled to
28x28.")
        from sklearn.datasets import load_digits
        from scipy.ndimage import zoom
        d = load_digits().images / 16.0
        d = np.stack([zoom(im, 3.5, order=1) for im in d]).reshape(-1, 784)
        x, xt = d[:int(0.8 * len(d))], d[int(0.8 * len(d)):]
    return (x[:n_train] > 0.5).astype(float), (xt[:n_test] > 0.5).astype(float)

X_train, X_test = load_mnist()
print("Train:", X_train.shape, "Test:", X_test.shape)

class RBM:
    def __init__(self, n_visible, n_hidden, seed=0):
        r = np.random.RandomState(seed)
        self.W = r.normal(0, 0.01, (n_visible, n_hidden))
        self.a = np.zeros(n_visible)        # visible bias
        self.b = np.zeros(n_hidden)         # hidden bias

    # ---- Task 1: energy E(v,h) = -a^T v - b^T h - v^T W h ----
    def energy(self, v, h):
        return -(v @ self.a) - (h @ self.b) - np.sum((v @ self.W) * h, axis=1)

    # free energy F(v) = -a^T v - sum_j log(1 + exp(b_j + v^T W_j))   (hidden units summed out)
    def free_energy(self, v):
        return -(v @ self.a) - np.sum(np.logaddexp(0, v @ self.W + self.b), axis=1)

    # ---- Task 2: conditional probabilities ----
    def p_h_given_v(self, v): return expit(v @ self.W + self.b)
    def p_v_given_h(self, h): return expit(h @ self.W.T + self.a)
```

```
    # ---- Task 3: one-step contrastive divergence (CD-1) ----
    def cd1(self, v0, lr):
        ph0 = self.p_h_given_v(v0)
        h0 = (rng.rand(*ph0.shape) < ph0).astype(float)          # sample hidden states
        pv1 = self.p_v_given_h(h0)                               # reconstruction
        ph1 = self.p_h_given_v(pv1)
        B = len(v0)
        self.W += lr * (v0.T @ ph0 - pv1.T @ ph1) / B            # positive phase - negative phase
        self.a += lr * np.mean(v0 - pv1, axis=0)
        self.b += lr * np.mean(ph0 - ph1, axis=0)

    def reconstruct(self, v):
        return self.p_v_given_h(self.p_h_given_v(v))

# ---------------- Sanity check: free energy = -log sum_h exp(-E(v,h)) on a tiny RBM ----------------
tiny = RBM(6, 3, seed=1); tiny.W = rng.normal(0, 1, (6, 3)); tiny.a = rng.normal(0, 1, 6); tiny.b =
rng.normal(0, 1, 3)
v_test = rng.randint(0, 2, (1, 6)).astype(float)
all_h = np.array(list(product([0, 1], repeat=3)), dtype=float)
brute = -np.log(sum(np.exp(-tiny.energy(v_test, h[None, :])[0]) for h in all_h))
print(f"\nBrute-force free energy = {brute:.6f} | formula = {tiny.free_energy(v_test)[0]:.6f}")

# ---------------- Task 3 & 4: training with CD-1; track free energy and reconstruction error ---------------
-
rbm = RBM(784, 64)
EPOCHS, BATCH, LR = 15, 100, 0.1
free_energy, recon_err = [], []
for ep in range(1, EPOCHS + 1):
    perm = rng.permutation(len(X_train))
    for i in range(0, len(X_train), BATCH):
        rbm.cd1(X_train[perm[i:i + BATCH]], LR)
    fe = rbm.free_energy(X_train[:2000]).mean()
    err = np.mean((X_train[:2000] - rbm.reconstruct(X_train[:2000])) ** 2)
    free_energy.append(fe); recon_err.append(err)
    print(f"Epoch {ep:2d}: avg free energy = {fe:8.2f} | reconstruction MSE = {err:.4f}")

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(range(1, EPOCHS + 1), free_energy, "o-"); ax[0].set_title("Average free energy");
ax[0].set_xlabel("Epoch")
ax[1].plot(range(1, EPOCHS + 1), recon_err, "o-", color="C1"); ax[1].set_title("Reconstruction error (MSE)");
ax[1].set_xlabel("Epoch")
plt.tight_layout(); plt.show()

test_err = np.mean((X_test - rbm.reconstruct(X_test)) ** 2)
print(f"\nTest reconstruction MSE = {test_err:.4f}")
print(f"Average energy E(v,h) of a test image with its most likely hidden state: "
      f"{rbm.energy(X_test[:200], (rbm.p_h_given_v(X_test[:200]) > 0.5).astype(float)).mean():.2f}")

# ---------------- Task 5: original vs reconstruction ----------------
n = 10
rec = rbm.reconstruct(X_test[:n])
fig, axes = plt.subplots(2, n, figsize=(15, 3.4))
for i in range(n):
    axes[0, i].imshow(X_test[i].reshape(28, 28), cmap="gray"); axes[0, i].axis("off")
    axes[1, i].imshow(rec[i].reshape(28, 28), cmap="gray"); axes[1, i].axis("off")
axes[0, 0].set_title("Original", loc="left"); axes[1, 0].set_title("Reconstructed", loc="left")
plt.tight_layout(); plt.show()
```

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Train: (10000, 784) Test: (1000, 784)

Brute-force free energy = -7.334564 | formula = -7.334564
Epoch  1: avg free energy =    -60.43 | reconstruction MSE = 0.0633
Epoch  2: avg free energy =    -81.16 | reconstruction MSE = 0.0541
Epoch  3: avg free energy =   -102.10 | reconstruction MSE = 0.0490
Epoch  4: avg free energy =   -115.74 | reconstruction MSE = 0.0459
Epoch  5: avg free energy =   -126.45 | reconstruction MSE = 0.0436
Epoch  6: avg free energy =   -137.46 | reconstruction MSE = 0.0419
Epoch  7: avg free energy =   -140.42 | reconstruction MSE = 0.0406
Epoch  8: avg free energy =   -149.14 | reconstruction MSE = 0.0394
Epoch  9: avg free energy =   -154.02 | reconstruction MSE = 0.0386
Epoch 10: avg free energy =   -160.56 | reconstruction MSE = 0.0378
Epoch 11: avg free energy =   -167.64 | reconstruction MSE = 0.0371
Epoch 12: avg free energy =   -170.29 | reconstruction MSE = 0.0366
Epoch 13: avg free energy =   -173.79 | reconstruction MSE = 0.0361
Epoch 14: avg free energy =   -178.67 | reconstruction MSE = 0.0356
Epoch 15: avg free energy =   -176.53 | reconstruction MSE = 0.0352
```



